



# Software Testing & Googletest

## Introduction

# Software Testing

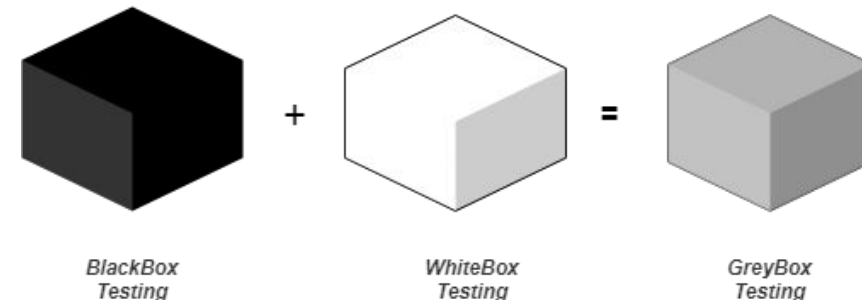
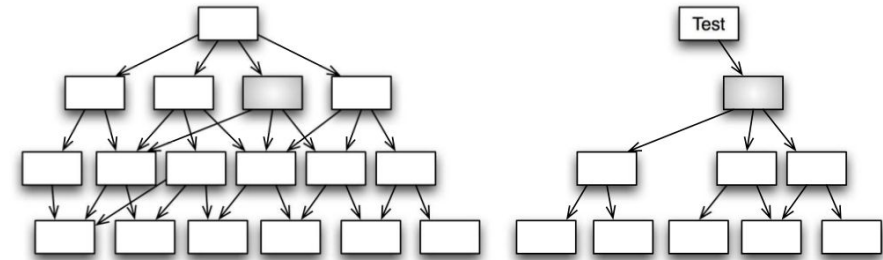
- ❖ If we want to make testable code we shall have modular code

- ❖ What is Software Testing?

- The process to evaluate functionalities of a software in order to
  - Find whether the developed software fulfils the specified requirements or not
  - Identify defects to ensure that the software is defect free

- ❖ Testing Methods:

- White Box Testing
- Black Box Testing
- Grey Box Testing
  - It is a combination of white-box testing and black-box testing
  - It is primarily used in Integration Testing



# Black Box Testing

---

- ❖ It is also known as **Behavioral** testing
- ❖ The internal structure/design/implementation of the component/system is unknown
- ❖ The higher the level, bigger and more complex the box, the more black-box testing is used
- ❖ It can be used in the functional and non-functional testings

- ❖ Some Techniques:

- Boundary Value Analysis

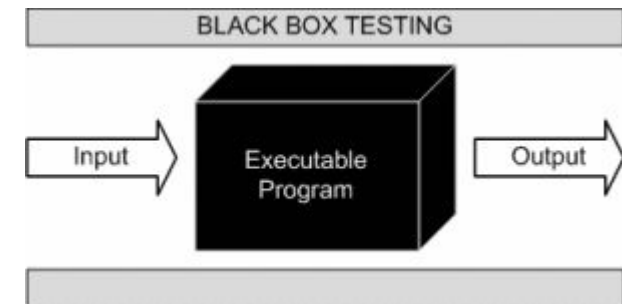
- Involves the determination of boundaries for input values
    - Using values that are at the boundaries and just inside/outside of the boundaries as test data

- Equivalence Partitioning

- Dividing input values into valid and invalid partitions and using them as the test data

- Cause-Effect Graphing:

- Involves identifying the cases that produce a **Cause-Effect Graph**, and generating test cases



# Black Box Testing

---

## ❖ Black Box Testing can be used to find errors in

- Incorrect or missing functions, Interface errors and errors in data structures or external database
- Behavior or performance errors, and initialization and termination errors

## ❖ Black Box Testing Advantages

- Tests are done from a user's point of view
- Tests can be done by a body independent of developers
- Tester does not need to know programming languages or how the software has been implemented
- Test cases can be designed as soon as the specifications are complete



[What is Black Box Testing?](#)

## ❖ Black Box Testing Disadvantages

- Only a small number of possible inputs can be tested and many program paths will be left untested
- Without clear specifications, which is the case in many projects, test cases will be difficult to design
- Tests can be redundant if the software designer/developer has already run a test case

# White Box Testing

---

- ❖ The internal structure/design/implementation of the component/system is known
- ❖ Testing is based on an analysis of the internal structure of the component or system
- ❖ White Box Testing Advantages
  - Testing is more complete, with the possibility of covering most paths
  - Code optimization by finding hidden errors
  - Testing can be started at an earlier stage
  - Test cases can be easily automated
- ❖ White Box Testing Disadvantages
  - It can be quite complex and expensive (highly skilled resources are required)
  - Test script maintenance can be a burden if the implementation changes too frequently
  - Since it is closely tied to the application under test, tools for every kind of implementation/platform may not be available



# White Box Testing

---

## ❖ White Box Testing Techniques

- Statement Coverage: Every possible statement in the code should be tested at least once during the testing
- Branch Coverage: It checks every possible path (if-else and other conditional loops) of a software

## ❖ What do you verify in White Box Testing?

- Internal security holes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object, and function on an individual basis
- Broken or poorly structured paths in the coding processes



[What is White Box Testing? Tutorial with Examples](#)

# Testing Levels

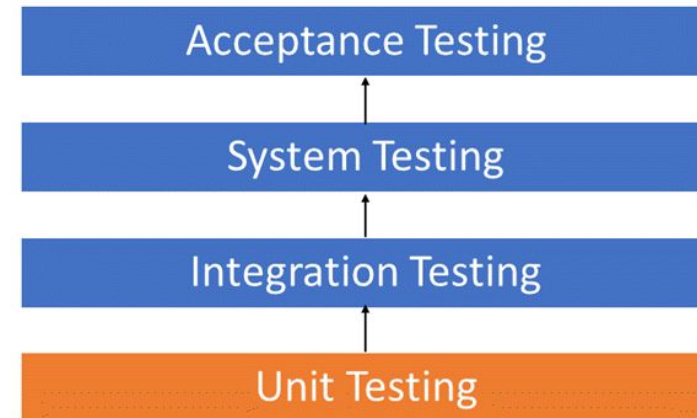
---

## ❖ Software Testing Levels

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

## ❖ Types of Software Testing

- Functional Testing
- Non-Functional Testing
- Maintenance Testing
- And etc.



[Types of Software Testing: 100 Examples of Different Testing Types](#)

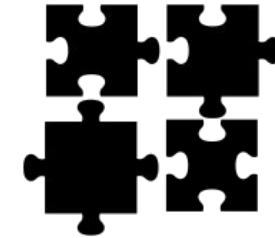
# Unit Testing

---

- ❖ The first level of testing; Individual units/components of a software are tested
- ❖ The purpose is to validate that each unit of the software performs as designed
- ❖ A unit is the smallest testable part of a software (e.g. functions, components and etc.)
- ❖ It is performed using the **White Box Testing** method
- ❖ It is normally performed by software developers themselves
- ❖ Unit Testing Benefits
  - It increases confidence in changing/maintaining code (they are run every time any code is changed)
  - Codes are more reusable. In order to make unit testing possible, codes need to be modular.
  - Development is faster (short term and long term)
    - Writing tests takes lesser time than the time it takes to run the tests
    - It takes lesser time to find and fix bugs than integration, system or acceptance testing
  - Debugging is easy. When a test fails, only the latest changes need to be debugged



[What is Unit Testing? - Software Testing Tutorial](#)

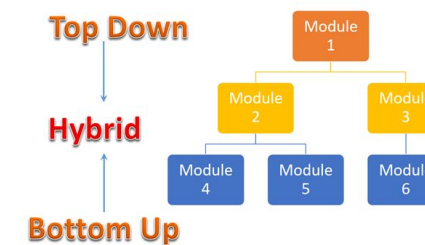
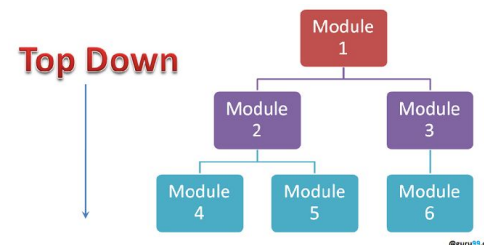
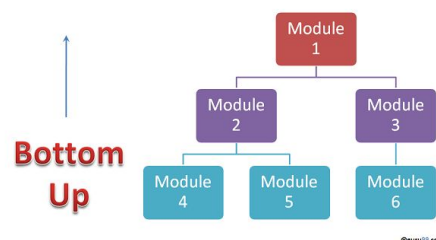
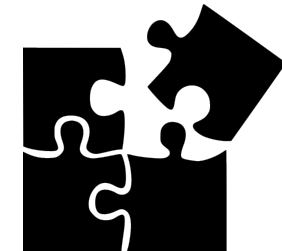


# Integration Testing

- ❖ It is a level of software testing where individual units are combined and tested as a group
- ❖ The purpose of this level of testing is to expose faults in the interaction between integrated units.
- ❖ Any of Black Box, White Box and Gray Box Testing methods can be used
- ❖ It is done by developers themselves or independent testers
- ❖ Integration Testing Approaches
  - **Big Bang:** Here all components are integrated together at once and then tested
  - **Incremental:** Testing is done by joining two or more modules that are logically related
    - Bottom-up Integration, Top-down Integration and Hybrid/ Sandwich Integration



[What is Integration Testing?](#)



# System and Acceptance Testing

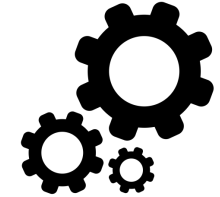
---

## ❖ System Testing



[What is System Testing?](#)

- The process of testing a fully integrated system to verify that it meets the system level requirements
- Usually, Black Box Testing method is used and it is done by Independent testers
- E.g. Usability Testing, Load Testing, Hardware/Software Testing and etc.



## ❖ Acceptance Testing

- Formal testing with respect to user requirements and business processes to determine whether or not a system satisfies the customer acceptance criteria
- Usually Black Box Testing is used. It is done by people who are not directly involved in the development of the product and can be done:
  - Internally: by members of the organization
  - External: by the customer or customer's of the customer (end users)

# GoogleTest

- ❖ A C++ unit testing framework and it is available across many platforms
- ❖ To download the framework visit [GoogleTest](#)
- ❖ Main features of GoogleTest
  - Test discovery
  - A rich set of assertions and user-defined assertions.
  - Death test, fatal and non-fatal failures
  - Value and Type parameterized tests.
  - And etc.
- ❖ You can build the framework using CMake.
  - Even CMake can generate the **main** function of test programs.
- ❖ **C++ TestMate** is a VS Code extension allowing to view GoogleTest in a tree view and run/debug your tests.

```
int main(int argc, char **argv) {  
    ::testing::InitGoogleTest(&argc, argv);  
    return RUN_ALL_TESTS();  
}
```



**C++ TestMate** v4.3.10

Mate Pek | 158,567 | ★★★★★ (16)

Run GoogleTest, Catch2 and DOCTest tests from VSCode

Disable

Uninstall



This extension is enabled globally.

# GoogleTest

- ❖ A **test case** is specification of inputs, execution condition, required steps and expected outcome of a single test.

```
TEST(TestSuiteName, TestName) {  
    ... test body ...  
}
```

- ❖ A collection of test cases executed in a specific test run is called a **test suite**.

- ❖ GoogleTest provides macros to test using assertions

- Failed assertions are reported.

- ❖ All assertion macros support streaming. For example:

- `EXPECT_TRUE(1 == 0) << "My condition is not true";`

- ❖ Fatal assertions aborts execution of their relative

test case. I.e. the rest of the body of **TEST()** are not be executed.

```
TEST(MyTestSuite, Test1) {  
    ASSERT_TRUE(1 == 0);  
    std::cout << "Test1 ..." << std::endl;  
}  
  
TEST(MyTestSuite, Test2) {  
    EXPECT_TRUE(1 == 0);  
    std::cout << "Test2 ..." << std::endl;  
}
```

| Fatal assertion                       | Nonfatal assertion                    | Verifies                        |
|---------------------------------------|---------------------------------------|---------------------------------|
| <code>ASSERT_TRUE(condition);</code>  | <code>EXPECT_TRUE(condition);</code>  | <code>condition</code> is true  |
| <code>ASSERT_FALSE(condition);</code> | <code>EXPECT_FALSE(condition);</code> | <code>condition</code> is false |

# GoogleTest

## ❖ Assertions Reference

### String Comparison

| Fatal Assertion                             | Nonfatal Assertion                          | Verifies                                                 |
|---------------------------------------------|---------------------------------------------|----------------------------------------------------------|
| <code>ASSERT_STREQ(expected, actual)</code> | <code>EXPECT_STREQ(expected, actual)</code> | the two C strings have the same content                  |
| <code>ASSERT_STRNE(str1, str2)</code>       | <code>EXPECT_STRNE(str1, str2)</code>       | the two C strings have different content                 |
| <code>ASSERT_STRCASEEQ(exp, act)</code>     | <code>EXPECT_STRCASEEQ(exp, act)</code>     | the two C strings have the same content, ignoring case   |
| <code>ASSERT_STRCASENE(str1, str2)</code>   | <code>EXPECT_STRCASENE(str1, str2)</code>   | the two C strings have different content, ignoring case. |

\* The assertions in this group compare two **C strings**. If you want to compare two *string* objects use `EXPECT_EQ`, etc.

\* CASE in an assertion name means that case is ignored.

### Floating Point Comparison

| Fatal Assertion                               | Nonfatal Assertion                            | Verifies                                                                               |
|-----------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------|
| <code>ASSERT_FLOAT_EQ(exp, act)</code>        | <code>EXPECT_FLOAT_EQ(exp, act)</code>        | The two float values are almost equal.                                                 |
| <code>ASSERT_DOUBLE_EQ(exp, act)</code>       | <code>EXPECT_DOUBLE_EQ(exp, act)</code>       | The two double values are almost equal                                                 |
| <code>ASSERT_NEAR(val1, val2, abs_err)</code> | <code>EXPECT_NEAR(val1, val2, abs_err)</code> | The difference between the <i>val1</i> and <i>val2</i> doesn't exceed <i>abs_err</i> . |

\* By "almost equal" we mean the two values are within 4 ULP's from each other.

### Binary Comparison

| Fatal Assertion                          | Nonfatal Assertion                       | Verifies                        |
|------------------------------------------|------------------------------------------|---------------------------------|
| <code>ASSERT_EQ(expected, actual)</code> | <code>EXPECT_EQ(expected, actual)</code> | <code>expected == actual</code> |
| <code>ASSERT_NE(val1, val2)</code>       | <code>EXPECT_NE(val1, val2)</code>       | <code>val1 != val2</code>       |
| <code>ASSERT_LT(val1, val2)</code>       | <code>EXPECT_LT(val1, val2)</code>       | <code>val1 &lt; val2</code>     |
| <code>ASSERT_LE(val1, val2)</code>       | <code>EXPECT_LE(val1, val2)</code>       | <code>val1 &lt;= val2</code>    |
| <code>ASSERT_GT(val1, val2)</code>       | <code>EXPECT_GT(val1, val2)</code>       | <code>val1 &gt; val2</code>     |
| <code>ASSERT_GE(val1, val2)</code>       | <code>EXPECT_GE(val1, val2)</code>       | <code>val1 &gt;= val2</code>    |

# GoogleTest

- ❖ A [death test](#) is any test which checks that a program is terminated in an expected fashion.  
Look at the [Death Assertions](#) macros.
- ❖ End name of your **test suite** (not test) with **DeathTest** when it contains a death test
- ❖ These assertions spawn new processes and execute the code under test in th processes.

```
int main(int argc, char *argv[])
{
    testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

```
#include <gtest/gtest.h>

void func(int x)
{
    if (x > 0) {
        std::cerr << "Bad Thing Happened!" << std::endl;
        exit(1); // Note that exit(0) is not a death
    }
}

TEST(example_DeathTest, SimpleDeath)
{
    EXPECT_DEATH(func(10), "Bad Thing.*");
    ASSERT_DEATH(func(10), "Bad Thing.*");
}

TEST(exampleDeathTest, SimpleExit)
{
    EXPECT_EXIT(func(10), ::testing::ExitedWithCode(1), "Bad Thing.*");
    ASSERT_EXIT(func(10), ::testing::ExitedWithCode(1), "Bad Thing.*");
}
```

# GoogleTest

- ❖ Predicate & Assertion Results
- ❖ Predicates provide detailed reports.
- ❖ A predicate Assertion verifies that the predicate **pred** returns true when passed the given values as arguments. **pred** is a function or functor that accepts as many arguments as the corresponding macro accepts values. If pred returns true for the given arguments, the assertion succeeds, otherwise the assertion fails.

| Fatal assertion                 | Nonfatal assertion              | Verifies                  |
|---------------------------------|---------------------------------|---------------------------|
| ASSERT_PRED1(pred1, val1)       | EXPECT_PRED1(pred1, val1)       | pred1(val1) is true       |
| ASSERT_PRED2(pred2, val1, val2) | EXPECT_PRED2(pred2, val1, val2) | pred2(val1, val2) is true |
| ...                             | ...                             | ...                       |

```
bool func(int m, int n) { return (m % n == 0); }

testing::AssertionResult isDivisible(int m, int n) {
    return func(m, n) ? (testing::AssertionSuccess() << m << " and " << n << " are divisible")
        : (testing::AssertionFailure() << m << " and " << n << " are not divisible");
}

TEST(MyTestSuite, ExpectTest) {
    EXPECT_FALSE(func(10, 5)); // Fails; less info
    EXPECT_FALSE(isDivisible(10, 5)); // Fails
    EXPECT_TRUE(isDivisible(10, 5)); // Succeeds
}
```

```
bool func(int m, int n) {
    return (m % n == 0);
}

TEST(MyTestSuite, ExpectTest) {
    EXPECT_FALSE(func(10, 5)); // Fails
}

TEST(MyTestSuite, PredicateTest) {
    EXPECT_PRED2(func, 10, 5); // Succeeds
    ASSERT_PRED2(func, 10, 6); // Fails
}
```

# GoogleTest

---

- ❖ Test Fixtures: Using the Same Data Configuration for Multiple Tests
  - Setting up the same data/objects for multiple tests.
- ❖ To create a fixture, derive a class from `::testing::Test`. Start its body with `protected:`,
  - We want to access fixture members from sub-classes.
- ❖ For each test the fixture object is created, its constructor and **SetUp** function are invoked, then the assertions are evaluated. At the end of the test, the **TearDown** function and the fixture object's destructor are called. The same procedure occurs for the next test.
- ❖ Use **override** keyword on **TearDown** and **SetUp** methods.
- ❖ When using a fixture, use **TEST\_F()** instead of **TEST()** as it allows you to access objects and subroutines in the test fixture.

```
TEST_F(TestFixtureName, TestName) {  
    ... test body ...  
}
```

# GoogleTest - Fixture Example

```
class stack {
public:
    static constexpr int SIZE{10};
public:
    bool push(int value) {
        bool status = false;
        if (top < SIZE - 1) { top++; status = true; array[top] = value; }
        return status;
    }

    bool pop(int &elem) {
        bool status = false;
        if (top > -1) { elem = array[top]; top--; status = true; }
        return status;
    }

    int available(void) { return (top + 1); }

    void clear(void) { top = -1; }
private: int array[SIZE]{0}; int top{-1};
};
```

```
class stackTest : public testing::Test {
protected: stack stck;
    void SetUp(void) override
    { for (int i = 1; i <= stack::SIZE; i++) { (void)stck.push(i); } }
    void TearDown(void) override {}
};

TEST_F(stackTest, PopTest) {
    int value, exp = stack::SIZE;
    while (stck.pop(value)) { ASSERT_EQ(value, exp); exp--; }
}

TEST_F(stackTest, ClearTest) {
    int value; stck.clear();
    ASSERT_FALSE(stck.pop(value));
    ASSERT_EQ(0, stck.available());
}

int main(void) {
    testing::InitGoogleTest();
    return RUN_ALL_TESTS();
}
```

# GoogleTest

- ❖ Typed tests allow you to repeat the same test logic over a list of types.
- ❖ First, define a fixture class template. E.g. `template <typename T> class FooTest : public testing::Test { /* ... */};`
- ❖ Next, associate a list of types with the test suite, which will be repeated for each type  
`using MyTypes = ::testing::Types<char, int, unsigned int>; TYPED_TEST_SUITE(FooTest, MyTypes);`
- ❖ The type alias (`using` or `typedef`) is necessary for the **TYPED\_TEST\_SUITE** macro.
- ❖ Then, use **TYPED\_TEST()** instead of **TEST\_F()** to define a typed test for this test suite.

```
#include <vector>
#include <gtest/gtest.h>

template <typename T> T largest(const std::vector<T> vect) { T res{};
    for (const auto &elem : vect) { if (res < elem) { res = elem; } } return res;
}

template <typename T> class LargestTest : public testing::Test {
protected: std::vector<T> vect;
void SetUp(void) override { T x{1}; for (int i = 0; i < 10; i++) { vect.push_back(i * x); } }
void TearDown(void) override {}
};
```

```
using MyTypes = ::testing::Types<float, int, unsigned int>;
TYPED_TEST_SUITE(LargestTest, MyTypes);

TYPED_TEST(LargestTest, testEmptyList) {
    std::vector<TypeParam> v; EXPECT_EQ(0, largest(v));
}

TYPED_TEST(LargestTest, testAList) {
    EXPECT_EQ(this->vect.back(), largest(this->vect));
}

int main(void) { ::testing::InitGoogleTest(); return RUN_ALL_TESTS(); }
```

# GoogleTest

- ❖ Type-parameterized tests are like typed tests
  - Except that they don't have to be defined at the time we write the TYPED\_TEST\_P(s).
- ❖ First, Define a fixture class template like typed tests
- ❖ Next, Declare a type-parameterized test suite. E.g. `TYPED_TEST_SUITE_P(LargestTest);`
- ❖ Then, use `TYPED_TEST_P()` to define a type-parameterized test.
- ❖ Then register all test patterns using the `REGISTER_TYPED_TEST_SUITE_P` macro
- ❖ Finally, you are free to instantiate the pattern with the types you want.

```
#include <vector>
#include <gtest/gtest.h>
template <typename T> T largest(const std::vector<T> vect) { T res{};
    for (const auto &elem : vect) { if (res < elem) { res = elem; } } return res; }
template <typename T> class LargestTest : public testing::Test {
protected: std::vector<T> vect; void TearDown(void) override {}
void SetUp(void) override { T x{1}; for (int i = 0; i < 10; i++)
    { vect.push_back(i * x); } }
};
TYPED_TEST_SUITE_P(LargestTest);
```

```
TYPED_TEST_P(LargestTest, testEmptyList) {
    std::vector<TypeParam> v; EXPECT_EQ(0, largest(v)); }
TYPED_TEST_P(LargestTest, testAList) {
    EXPECT_EQ(this->vect.back(), largest(this->vect)); }

REGISTER_TYPED_TEST_SUITE_P(LargestTest, testEmptyList, testAList);
using MyTypes = ::testing::Types<float, int, unsigned int>;
INSTANTIATE_TYPED_TEST_SUITE_P(My, LargestTest, MyTypes);

int main(void) { ::testing::InitGoogleTest();
return RUN_ALL_TESTS(); }
```

# GoogleTest

---

- ❖ [Value-parameterized tests](#) allow you to test your code with different parameters without writing multiple copies of the same test.
- ❖ First, define a fixture class. Derive the fixture class from **testing::TestWithParam<T>**
  - Where T is the type of your parameter values and it can be any copyable type
- ❖ Then, use **TEST\_P** to define as many test patterns using this fixture as you want.
- ❖ Inside a test, you can access the test parameter with the **GetParam()**
- ❖ Finally, you can use the **INstantiate\_Test\_Suite\_P** macro to instantiate the test suite with any set of parameters you want.  

```
INstantiate_Test_Suite_P(UniqueInstantiationName, TestSuite, testing::Values(1, 2, 3));
```
- ❖ The first argument is a unique name for the instantiation, the second argument is the test suite and the last argument is a [parameter generator](#).

# GoogleTest

```
#include <gtest/gtest.h>

template <typename T> T addOne(T x) { return (x + 1); }

class AddOneTestsFixture : public testing::TestWithParam<int> {};

INSTANTIATE_TEST_SUITE_P(AddOneTests,
                          AddOneTestsFixture,
                          testing::Values(1, 2, 3, 4, 5));

TEST_P(AddOneTestsFixture, doAdd) {
    int input = GetParam();
    ASSERT_EQ(addOne(input), input + 1);
}

int main(void) {
    ::testing::InitGoogleTest();
    return RUN_ALL_TESTS();
}
```

```
#include <gtest/gtest.h>

template <typename T> T addOne(T x) { return (x + 1); }

template <typename T> class AddOneTestsFixture : public testing::Test {
    const std::tuple<std::vector<int>, std::vector<double>> allValues{
        {1, 2, 3, 4, 5, 6, 7, 8}, /* Test values for int */
        {1.5, 2.5, 3.5, 4.5, 5.5} /* Test values for double */
    };
protected:
    const std::vector<T> values{std::get<std::vector<T>>(allValues)};
};

using MyTypes = ::testing::Types<int, double>;
TYPED_TEST_SUITE(AddOneTestsFixture, MyTypes);

TYPED_TEST(AddOneTestsFixture, doAddOne) {
    for (auto const &elem : this->values) {
        ASSERT_EQ(addOne(elem), elem + 1);
    }
}
```

# Modules & Test Double

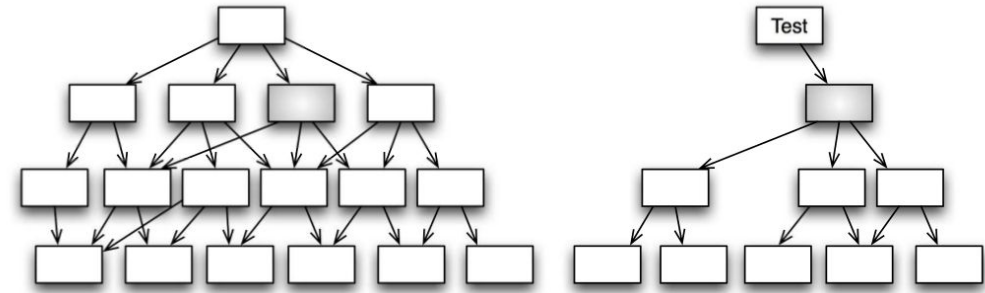
---

- ❖ **CUT** : Code Under Test
- ❖ A **dependency** is a function, data, module, library or device outside the CUT
  - Real code has dependencies. A module interacts with several others to do its job.
  - Dependencies make test automation difficult and expensive
- ❖ A **collaborator** is a dependency that the CUT depends upon
- ❖ A **test double** impersonates some dependencies during the test of the CUT
  - The CUT does not know it is using a test double
  - The CUT interacts with the double in the same way it interacts with the real collaborator
  - The test double is a stub taking the place of the actual production code
  - In TDD, the test doubles are used to facilitate automated testing

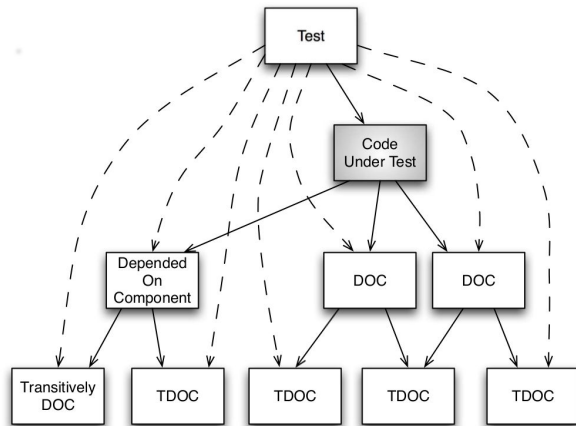
# Modules & Test Double

## ❖ Breaking Dependencies

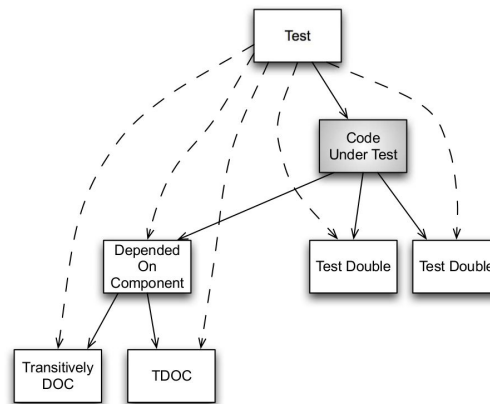
- Using of interfaces
- Using of hierarchical design
- Encapsulation and hiding details
- Less reliance on unprotected global data



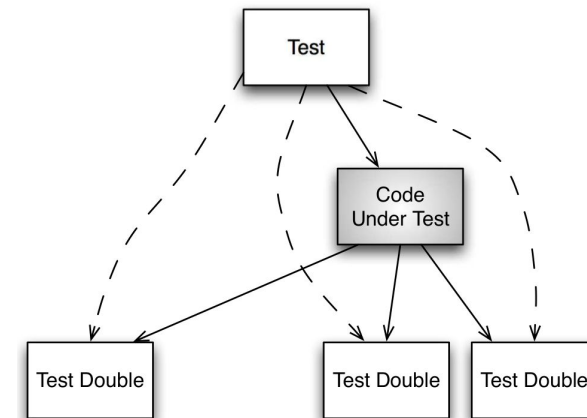
Dependencies of the gray module



Test Dependency Mess



Using test doubles and real collaborators



Manage test dependencies with test doubles

# Modules & Test Double

---

- ❖ The rule of thumb
  - Use the real code when you can and use a test double when you must
- ❖ When to Use a Test Double
  - Hardware independence
    - Testing independently from the hardware
    - Providing a wide variety of inputs into the core of the system that may be difficult to do it in the lab
  - Inject difficult to produce input(s)
    - Some computed or hardware-generated event scenarios may be difficult to produce
  - Speed up a slow collaborator
    - A slow collaborator, such as a database or a network service

# Modules & Test Double

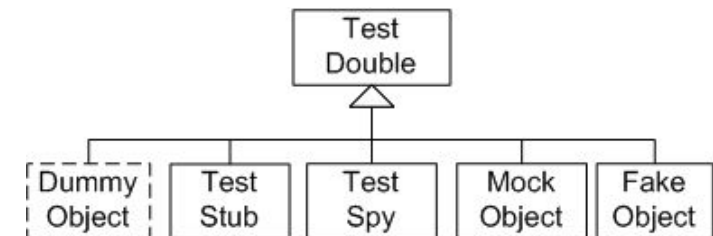
---

## ❖ When to Use a Test Double ...

- Dependency on something under development
- Dependency on something volatile
  - Like an alarm clock.
  - You have an event that is supposed to get triggered at 03:42
- Dependency on something that is difficult to configure
  - A database is an example of a DOC that you could test with but is difficult to set up

## ❖ Test Doubles Variations

- There are different types of doubles like
  - Test Stub, Test Spy, Fake Object, Mock Object and etc.



# Modules & Test Double Variations

## ❖ Test dummy

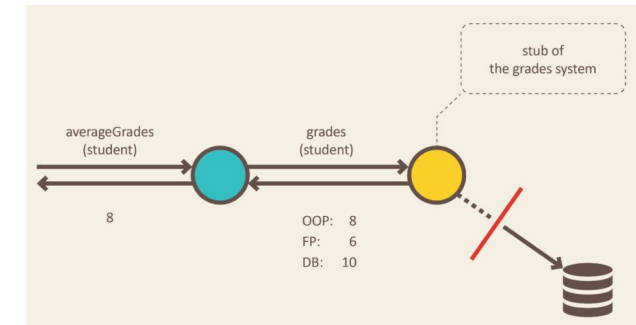
- It is provided to satisfy the compiler, linker or runtime dependency

## ❖ Test stub

- It returns a value, as directed by the current test case
- It holds predefined data and uses it to answer calls during tests
- It is used when we cannot or don't want to involve objects that would answer with real data

## ❖ Test spy

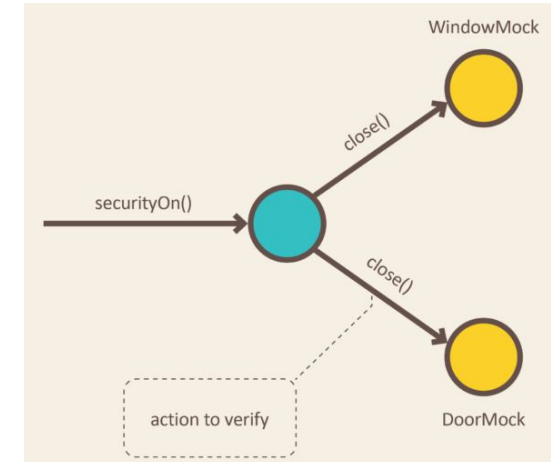
- It captures the parameters passed from the CUT so the test can verify that the correct parameters have been passed to the CUT
- It can also feed return values to the CUT just like a test stub



# Modules & Test Double Variations

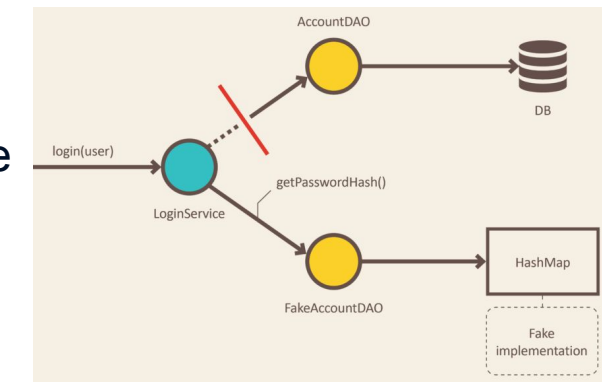
## ❖ Mock Object

- It Verifies function calls, call orders, and the parameters passed from the CUT to the DOC
- It usually deals with a situation where multiple calls are made to it, and each call and response are potentially different



## ❖ Fake Object

- It provides a partial and simplified version of the production code
- It is used when the CUT depends on a component which makes testing difficult or slow



- ❖ In C++ OOP, you can use virtualization and link-time substitution in order to make test doubles

# GoogleTest - gMock

## ❖ How can we test the Today class?

```
#include <ctime>
#include <chrono>
#include <gtest/gtest.h>

class Date {
    int day, month, year;

public:
    Date() {
        std::chrono::system_clock::time_point now = std::chrono::system_clock::now();
        std::time_t time_temp = std::chrono::system_clock::to_time_t(now);
        const std::tm *local_tm = std::localtime(&time_temp);
        year = local_tm->tm_year + 1900;
        month = local_tm->tm_mon + 1;
        day = local_tm->tm_mday;
    }

    int getYear(void) { return year; }
    int getMonth(void) { return month; }
    int getDay(void) { return day; }
};
```

```
class Today {
public: bool isLeapDay(void) {
    Date date; bool status = false;
    int day = date.getDay(); int year = date.getYear();
    int month = date.getMonth();
    int days[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

    if ((day > days[month - 1]) && (month == 2)) {
        if ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0))) {
            if (day == days[month - 1] + 1) { status = true; }
        }
    }

    return status;
}
};

TEST(TodayTest, leapDay)
{ Today today; EXPECT_FALSE(today.isLeapDay()); /* ??? */ }

int main(void) {
    ::testing::InitGoogleTest();
    return RUN_ALL_TESTS();
}
```

# GoogleTest - gMock

---

- ❖ A **mock** object implements the same interface as a real object
- ❖ **Mocks** are objects pre-programmed with expectations, which form a specification of the calls they are expected to receive.
- ❖ **Fake** objects have working implementations, but usually take some shortcut , which makes them not suitable for production.
- ❖ Dummies, Stubs, Spies, and Mocks could be auto generated.
- ❖ [gMock](#) is a library to create mock classes and use them. gmock/gmock\_main shall be linked
- ❖ When using gMock:
  - First, use some macros to describe the interface you want to mock, They will be expanded to the implementation of the mock class
  - Next, create some mock objects and specify expectations on them
  - Then, test the code that uses the mock objects. gMock will catch any violation to the expectations as soon as it arises.

# GoogleTest - gMock

- ❖ The Today class depends on Now, To test it we need to make an interface for Now. E.g. INow

```
struct INow { virtual int getYear(void) = 0; virtual int getMonth(void) = 0; virtual int getDay(void) = 0; };
```

- ❖ We need to create a class, for example Now, based on INow and override the pure virtual functions in INow and implement them.

```
class Now : public INow{ ... };
```

- ❖ The Today class shall have a variable of type INow \* or INow & and a constructor to initialize it.
- ❖ When we create an instance of Today we can pass a pointer to an instance of any class based on INow to the constructor.

- In production mode we create an instance of **Now** and pass its address to the Today class constructor
- In test mode we can make a **mock for the Now class** and create an instance of it and pass its address to the Today class constructor
- Using [gMock](#) we can create the mock

```
class Today {  
    INow *now{nullptr};  
public:  
    Today(INow *nw) : now{nw} {}  
    bool isLeapDay(void);  
};
```

# GoogleTest - gMock

- ❖ To define the mock create a class based on INow
- ❖ In the public section use MOCK\_METHOD(); macro
  - This macro will generate the definitions for you
- ❖ Take the function prototype, cut-and-paste it into the macro, and add two commas - one between the return type and the name, another between the name and the argument list.

```
class NowMock : public INow {  
public:  
    MOCK_METHOD(int, getYear, (), (override));  
    MOCK_METHOD(int, getMonth, (), (override));  
    MOCK_METHOD(int, getDay, (), (override));  
};
```

```
class Turtle {  
    ...  
    virtual ~Turtle() {}  
    virtual void PenUp() = 0;  
    virtual void PenDown() = 0;  
    virtual void Forward(int distance) = 0;  
    virtual void Turn(int degrees) = 0;  
    virtual void GoTo(int x, int y) = 0;  
    virtual int GetX() const = 0;  
    virtual int GetY() const = 0;  
};
```

```
class MockTurtle : public Turtle {  
public:  
    ...  
    MOCK_METHOD(void, PenUp, (), (override));  
    MOCK_METHOD(void, PenDown, (), (override));  
    MOCK_METHOD(void, Forward, (int distance), (override));  
    MOCK_METHOD(void, Turn, (int degrees), (override));  
    MOCK_METHOD(void, GoTo, (int x, int y), (override));  
    MOCK_METHOD(int, GetX, (), (const, override));  
    MOCK_METHOD(int, GetY, (), (const, override));  
};
```

# GoogleTest - gMock

- ❖ To use the mock, you can import the gMock names from the testing namespace
- ❖ Create some mock objects and specify your expectations on them (How many times will a method be called? With what arguments? What should it do? etc.).
- ❖ Exercise some code that uses the mocks; optionally, check the result using gtest assertions.
- ❖ When a mock is destructed, gMock will check if all expectations on it have been satisfied.
- ❖ **Important note:** gMock requires expectations to be set before the mock functions are called, otherwise the behavior is undefined.
- ❖ **EXPECT\_CALL()** should be read as expecting that a call will occur in the future, not that a call has occurred.

```
using ::testing::Return;

TEST(TodayTest, TestLeap) {
    NowMock now; Today today{&now};
    EXPECT_CALL(now, getYear()).WillOnce(Return(2023));
    EXPECT_CALL(now, getMonth()).WillOnce(Return(4));
    EXPECT_CALL(now, getDay()).WillOnce(Return(29));
    EXPECT_FALSE(today.isLeapDay());

    EXPECT_CALL(now, getYear()).WillOnce(Return(2024));
    EXPECT_CALL(now, getMonth()).WillOnce(Return(2));
    EXPECT_CALL(now, getDay()).WillOnce(Return(29));
    EXPECT_TRUE(today.isLeapDay());
}
```

# GoogleTest - gMock

- ❖ **EXPECT\_CALL()** macro to set an expectation on a mock method

➤ For the details, look at [Mocking Reference](#)

- ❖ If a mock method has no EXPECT\_CALL spec but is called, we say that it's an **"uninteresting call"**, and the default action (which can be specified using [ON\\_CALL\(\)](#)) of the method will be taken.
- ❖ An uninteresting call will also by default cause gMock to print a warning. It is possible to change the default behaviour using [The Nice and the Strict](#) mock

```
EXPECT_CALL(mock_object, method_name(matchers...))  
  .With(multi_argument_matcher) // Can be used at most once  
  .Times(cardinality)           // Can be used at most once  
  .InSequence(sequences...)    // Can be used any number of times  
  .After(expectations...)       // Can be used any number of times  
  .WillOnce(action)             // Can be used any number of times  
  .WillRepeatedly(action)       // Can be used at most once  
  .RetiresOnSaturation();       // Can be used at most once
```

```
ON_CALL(mock_object, method_name(matchers...))  
  .With(multi_argument_matcher) // Can be used at most once  
  .WillByDefault(action);       // Required
```

```
using ::testing::NiceMock;  
using ::testing::StrictMock;
```

```
MockFoo mock_foo; // Generate warnings  
NiceMock<MockFoo> mock_foo; // It suppresses warnings  
StrictMock<MockFoo> mock_foo; // It makes all uninteresting calls failures
```

# Software Testing & Googletest

---

## ❖ Some useful links

- [Effective Tests: Test Doubles](#)
- [Test Doubles — Fakes, Mocks and Stubs](#)
- [Test Double](#)
- [Google Test and CMake](#)
- [GoogleTest User's Guide](#)